TOPIC 00.1: WELCOME & SIGNATURE WORKFLOW

💡 **OUR COACHING PHILOSOPHY: HOW TO LEARN, NOT WHAT TO MEMORIZE**

Most DSA books teach solutions to individual problems. Our system teaches **pattern recognition and algorithm derivation**. You will learn to recognize clues in the question, pick the right pattern, recall the generic template, and write clean, production-grade Java code.

🎯 OUR SIGNATURE 10-STEP WORKFLOW

1. Read Problem Carefully
↓
2. Find Clues & Keywords in Statement
↓
3. Recognize Algorithmic Pattern
↓
4. Recall Generic Code Template
↓
5. Customize Template for Problem
↓
6. Dry Run with Concrete Example
↓
7. Write Production Java Code
↓
8. Analyze Time & Space Complexity
↓
9. Verify Edge Cases (null, 0, 1, max N)
↓
10. Proactively Handle Interviewer Follow-ups

🧠 SENIOR ENGINEER MENTOR STYLE

- ✅ **Derive First:** Always build brute force first, then optimize step-by-step.
- ✅ **Draw Before Coding:** Visualize pointers, nodes, matrices, and state transitions.
- ✅ **Aha Moment 💡:** Every topic unlocks ONE core mental shift.
- ⚠️ **No Black Boxes:** Understand standard Java collections (HashMap treeification, PriorityQueue binary heap, ArrayDeque circular array).
- ✅ **Defensive Guards:** Always handle `null`, empty arrays, and single-element inputs at the very top.

📌 **GOLDEN RULE:** Master the pattern, master any problem. Memorizing 300 problems leads to failure on unseen interview questions; mastering 20 core templates guarantees success.

TOPIC 00.2: THE 13-PHASE FAANG LEARNING FRAMEWORK

🧠 THE 13-PHASE LEARNING PIPELINE (APPLIED TO EVERY TOPIC)

Phase	Name	Goal	Activity
Phase 0	🎯 Mindset	WHY does this topic exist?	Real-world uses, FAANG frequency, motivation
Phase 1	💡 Intuition First	Feel the pattern	Analogies, stories, visualizations, scratch building
Phase 2	📖 Core Understanding	Definitions & Rules	Properties, invariants, edge cases, common traps
Phase 3	🎯 Pattern Recognition	Spot the clues	Keyword mapping, signal detection, when NOT to use
Phase 4	🔨 Build Algorithm	Derive together	Brute force ➔ Bottleneck ➔ Stepwise optimization
Phase 5	⚙️ Template Building	Reusable code	Generic algorithm ➔ Java template skeleton
Phase 6	▶️ Dry Run	Variable movement	Small & medium traces, state transition tables
Phase 7	💻 Coding Together	Line-by-line Java	Explain every variable, loop condition, and guard
Phase 8	📊 Complexity Thinking	Time & Space O()	Asymptotic analysis, memory/speed trade-offs
Phase 9	🦎 Debugging Mindset	Catch bugs early	Off-by-one errors, null checks, dry-run mistakes
Phase 10	🔁 Reverse Thinking	Handle twists	Interviewer constraint changes & follow-ups
Phase 11	📈 Pattern Expansion	Progressive practice	Easy ➔ Medium ➔ Hard ➔ Combined patterns
Phase 12	✅ Mastery Test	Teach it back	Explain without notes, write template from memory

💡 Every topic in Book 1 follows this exact 13-phase structure so your learning is consistent, structured, and deep!

TOPIC 00.3: REUSABLE UTILITIES & DATA CONVERSIONS

🔄 UNIVERSAL DATA CONVERSIONS CHEAT SHEET

1. Arrays & Collections Conversions

```
// int[] ➔ List<Integer>
List<Integer> list =
Arrays.stream(arr).boxed().collect(Collectors.toList());

// List<Integer> ➔ int[]
int[] arr = list.stream().mapToInt(i → i).toArray();

// Set ➔ int[]
int[] arr = set.stream().mapToInt(Number::intValue).toArray();
```

2. String & Character Conversions

```
// String ➔ char[]
char[] chars = str.toCharArray();

// char[] ➔ String
String s = new String(chars);

// Character frequency array (a-z)
int[] freq = new int[26];
freq[ch - 'a']++;
```

3. Graph & Grid Utilities

```
// Grid Direction Vectors (4-way & 8-way)
int[][] DIR4 = {{-1,0}, {1,0}, {0,-1}, {0,1}};
int[][] DIR8 = {{-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},
{1,1}};

// Grid Boundary Check
boolean isValid(int r, int c, int R, int C) {
    return r ⩾ 0 && r < R && c ⩾ 0 && c < C;
}
```

4. Edge List ➔ Adjacency List

```
List<List<Integer>> buildGraph(int n, int[][] edges) {
    List<List<Integer>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] e : edges) {
        g.get(e[0]).add(e[1]);
        g.get(e[1]).add(e[0]); // remove for directed
    }
    return g;
}
```

💡 **AHA:** Master these conversions first so you never waste interview minutes wrestling with syntax!

TOPIC 00.4: MASTER DEPENDENCY ROADMAP & STUDY PLANS

🗺️ 20-TOPIC DEPENDENCY ROADMAP

Part	Topics	Focus Area
Part 1	Topic 01 – 05	Foundations, Arrays, Two Pointers, Sliding Window, Binary Search
Part 2	Topic 06 – 09	Linked Lists, Stack, Queue & Deque, Heap / Priority Queue
Part 3	Topic 10 – 12	Trees & BST, Trie (Prefix Tree), Graphs Masterclass
Part 4	Topic 13 – 16	Backtracking, Dynamic Programming, Greedy Algorithms, Intervals
Part 5	Topic 17 – 19	Bit Manipulation, Mathematics, Advanced Data Structures
Book 2	Topic 20 – 30	FAANG Interview Mastery Playbook & Mock Interviews

🗓️ 4-WEEK INTENSIVE STUDY SCHEDULE

Week 1: Foundations, Arrays, Two Pointers, Sliding Window, Binary Search
Week 2: Linked Lists, Stacks, Queues, Heaps, Trees & BST
Week 3: Tries, Graphs, Backtracking, Dynamic Programming
Week 4: Greedy, Intervals, Bitmask, Math, Advanced DS & Book 2 Mocks

🏆 SUCCESS CRITERIA

- ✅ Solve any Medium in < 20 min
- ✅ Solve any Hard in < 35 min
- ✅ Write bug-free Java code with guards
- ✅ Narrate thinking out loud continuously

🚀 You are now ready to begin! Head to **Topic 01: Foundations & Big-O Masterclass**.